

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
IMAGE_PATH = "/content/big-white-heron-decreased-contrast.jpg"
```

```
img = cv2.imread(IMAGE_PATH, cv2.IMREAD_GRAYSCALE)
```

```
if img is None:
    print(f"Could not find '{IMAGE_PATH}'. Generating a synthetic low-contrast image instead.")
```

```
base = np.random.normal(loc=125, scale=8, size=(300, 300))
xx, yy = np.meshgrid(np.linspace(0, 4 * np.pi, 300), np.linspace(0, 4 * np.pi, 300))
pattern = 15 * np.sin(xx) * np.cos(yy)
img = np.clip(base + pattern, 100, 150).astype(np.uint8)
```

```
def show_image_and_histogram(image, title):
    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(image, cmap="gray", vmin=0, vmax=255)
    plt.title(title)
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.hist(image.ravel(), bins=256, range=(0, 256), color="gray")
    plt.title(f"Histogram - {title}")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")

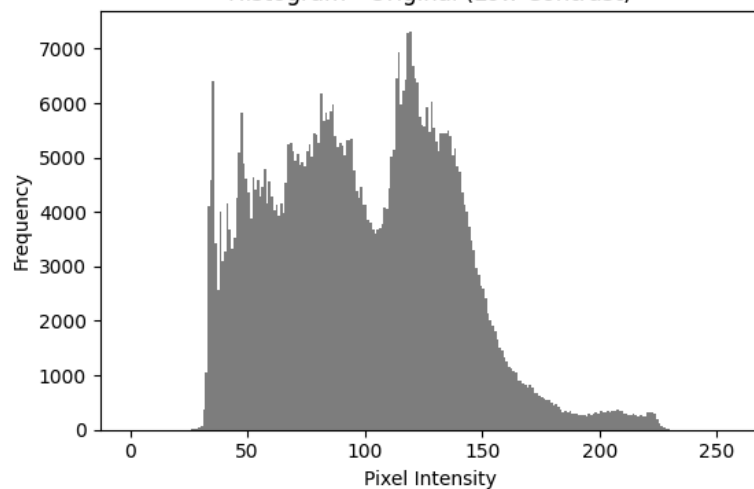
    plt.tight_layout()
    plt.show()
```

```
show_image_and_histogram(img, "Original (Low Contrast)")
```

Original (Low Contrast)



Histogram - Original (Low Contrast)



```
def contrast_stretching(image):
    """
    Linearly stretches pixel intensities to span the full 0-255 range:
    new_pixel = (pixel - min) * (255 / (max - min))
    """
    min_val = np.min(image)
    max_val = np.max(image)

    if max_val == min_val:
        return image.copy()

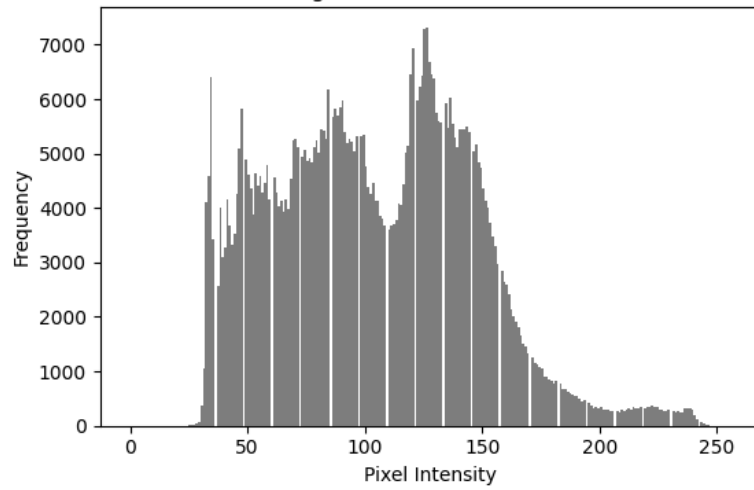
    stretched = (image.astype(np.float32) - min_val) * (255.0 / (max_val - min_val))
    stretched = np.clip(stretched, 0, 255).astype(np.uint8)
    return stretched
```

```
stretched_img = contrast_stretching(img)
show_image_and_histogram(stretched_img, "Contrast Stretched")
```

Contrast Stretched



Histogram - Contrast Stretched

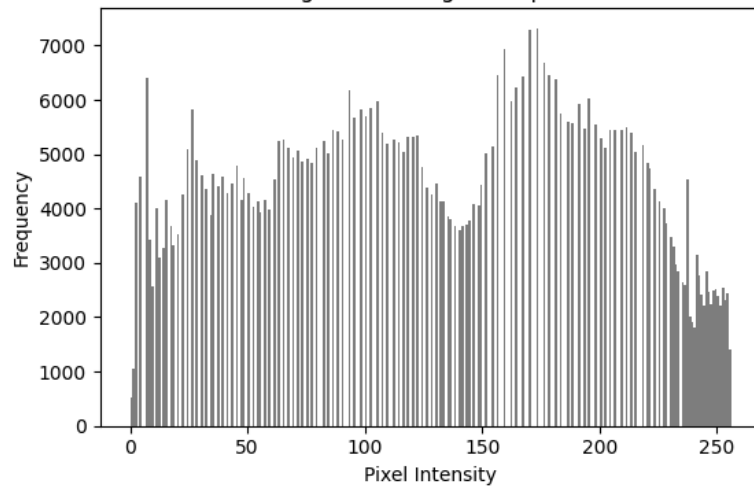


```
equalized_img = cv2.equalizeHist(img)
show_image_and_histogram(equalized_img, "Histogram Equalized")
```

Histogram Equalized



Histogram - Histogram Equalized

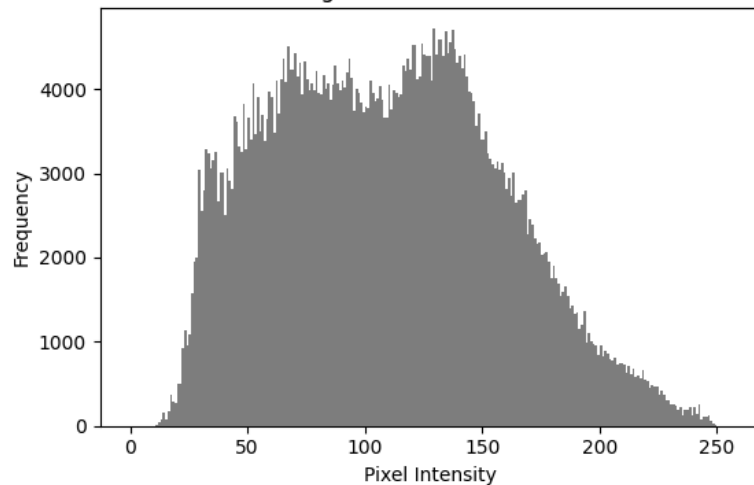


```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_img = clahe.apply(img)
show_image_and_histogram(clahe_img, "CLAHE Enhanced")
```

CLAHE Enhanced



Histogram - CLAHE Enhanced



```
titles = ["Original", "Contrast Stretched", "Histogram Equalized", "CLAHE"]
images = [img, stretched_img, equalized_img, clahe_img]
```

```
plt.figure(figsize=(16, 8))
for i in range(4):
    # Image
    plt.subplot(2, 4, i + 1)
    plt.imshow(images[i], cmap="gray", vmin=0, vmax=255)
    plt.title(titles[i])
    plt.axis("off")
```

Original



Contrast Stretched



Histogram Equalized



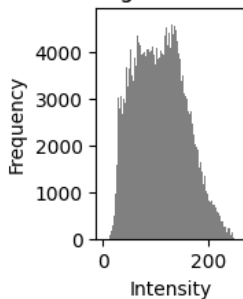
CLAHE



```
plt.subplot(2, 4, i + 5)
plt.hist(images[i].ravel(), bins=256, range=(0, 256), color="gray")
plt.title(f"Histogram - {titles[i]}")
plt.xlabel("Intensity")
plt.ylabel("Frequency")
```

```
Text(0, 0.5, 'Frequency')
```

Histogram - CLAHE



```
plt.tight_layout()
plt.savefig("comparison_output.png", dpi=150)
plt.show()
```

<Figure size 640x480 with 0 Axes>

```
print("Saved comparison figure as 'comparison_output.png'")
```

Saved comparison figure as 'comparison_output.png'

```
print("\n--- Quantitative Comparison (Standard Deviation as a proxy for contrast) ---")
for title, image in zip(titles, images):
    print(f"{title:22s} -> Mean: {np.mean(image):6.2f} | Std Dev (contrast): {np.std(image):6.2f}")
```

```
--- Quantitative Comparison (Standard Deviation as a proxy for contrast) ---
```

```
Original          -> Mean:  98.69 | Std Dev (contrast):  38.57
Contrast Stretched -> Mean: 103.76 | Std Dev (contrast):  42.04
Histogram Equalized -> Mean: 128.45 | Std Dev (contrast):  73.57
CLAHE             -> Mean: 108.49 | Std Dev (contrast):  48.35
```

```
cv2.imwrite("stretched_output.jpg", stretched_img)
cv2.imwrite("equalized_output.jpg", equalized_img)
cv2.imwrite("clahe_output.jpg", clahe_img)
```

True

```
print("\nSaved output images: stretched_output.jpg, equalized_output.jpg, clahe_output.jpg")
```

Saved output images: stretched_output.jpg, equalized_output.jpg, clahe_output.jpg

